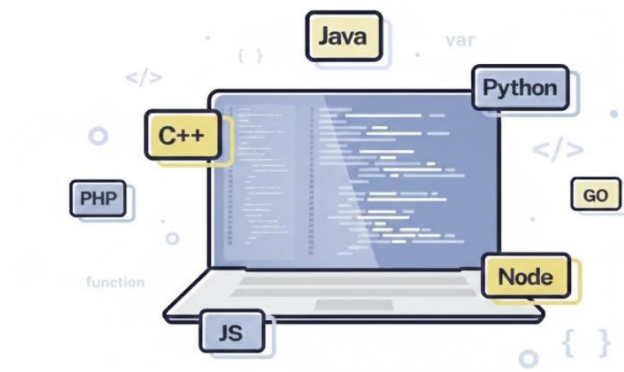


King Saud University
College of Computer and Information Sciences
Department of Information Technology
Semester-2 1447H
Submission Date: 10 May 2026



Beyond the Code: Multi-Dimensional Insights into the Global Developer Ecosystem

IT 462- Big Data Systems Final Project Report

Group #5 Section: 2766	Students Name	ID	Email
	Rose Mady	444200107	444200107@student.ksu.edu.sa
	Layan Alfawzan	444200793	444200793@student.ksu.edu.sa
	Aljawharah Alwabel	444200750	444200750@student.ksu.edu.sa
	Layan Aldbays	444200653	444200653@student.ksu.edu.sa

Supervised by: Dr. Afshan Jafari

Table of Contents

1 Introduction	4
1.1 Background and Context	4
1.2 Problem Statement.....	4
1.3 Objectives	4
1.4 Scope and Limitations	4
2 Data Selection and Understanding.....	5
2.1 Dataset Description.....	5
2.2 Structure and Schema	5
2.3 Initial Data Quality Observations	6
2.4 Ethical and Legal Considerations	6
3 Data Preprocessing	6
3.1 Cleaning.....	6
3.2 Integration.....	7
3.3 Reduction.....	7
3.4 Transformation	7
3.5 Summary of Preprocessing Pipeline.....	7
4 RDD Operations	8
4.1 RDD Design	8
4.2 Key Transformations and Actions	8
4.3 Reflection on RDD Usage	10
5 SQL Operations	10
5.1 DataFrame and View Creation	10
5.2 Key Analytical Queries.....	10
5.3 Summary of Insights from SQL	13
6 Machine Learning Operations	13
6.1 Problem Formulation	13
6.2 Feature Pipeline	14
6.3 Model Choice	14
6.4 Training and Evaluation Setup	14
6.5 Results	14
6.6 Interpretation	14
6.7 Limitations.....	16

7 Conclusion and Future Work.....	16
7.1 Summary of Contributions	16
7.2 Reflection.....	16
7.3 Future Work.....	17
8 References.....	17
9 Appendix	17

List of Figures

Figure 1. Language Popularity Across the Developer Ecosystem Output.....	8
Figure 2. Salary Distribution by Developer Role Output.	9
Figure 3. High Earners and Above-Average Salary Filter Output.....	9
Figure 4. DataFrame and View Creation.	11
Figure 5. Average Salary by Developer Type Query Result.	11
Figure 6. Top Programming Languages by Developer Type Query Result.	12
Figure 7. Global Talent Distribution Query Result.	12
Figure 8. Random Forest Confusion Matrix.....	15
Figure 9. Random Forest Feature Importance.	15

List of Tables

Table 1. Stack Overflow 2018 Developer Survey Dataset Schema.....	5
Table 2. Data Cleaning Summary.....	6
Table 3. RDD Analyses Summary.....	8
Table 4. SQL Query Insights Summary.	10
Table 5. Developer Type Class Distribution.	13
Table 6. ML Input Features.	13
Table 7. Model Evaluation Results.....	14
Table 8. Per-Class Performance.	15

1 Introduction

1.1 Background and Context

This project is situated within the software development and workforce analytics domain. It analyzes the Stack Overflow 2018 Developer Survey, accessed through Kaggle [1]. The dataset captures global responses from software developers, including information about education, employment status, technical skills, salaries, and professional roles.

In recent years, the technology industry has experienced rapid transformation driven by the rise of data science, artificial intelligence, cloud computing, and advanced analytics. As a result, many developers have transitioned from traditional roles such as web or back-end development toward specialized positions such as Data Scientist or Machine Learning Engineer. Understanding these shifts is important for companies, recruiters, universities, and training institutions that need to align workforce planning and curricula with market demand.

By analyzing educational background, current expertise, and future technology preferences, this study provides data-driven insights into developer career trajectories and workforce trends within the global technology ecosystem.

1.2 Problem Statement

Many developers are moving from traditional software roles toward emerging data-oriented roles. This project analyzes the Stack Overflow 2018 Developer Survey to identify how educational background, employment status, experience, salary, and technical skills relate to a developer's role. The main task is to predict the developer segment while also explaining meaningful patterns discovered through RDD, SQL, and machine learning operations. The target stakeholders include technology recruiters, educational institutions, workforce planners, and learners interested in developer career pathways.

1.3 Objectives

- Clean, transform, and reduce a large-scale developer survey dataset into a modeling-ready structure.
- Explore key patterns in developer roles, salaries, countries, educational backgrounds, and programming languages using RDD and SQL operations.
- Build and evaluate supervised classification models that predict DevType_segment from professional and educational features.
- Interpret model outputs and communicate insights about the global developer ecosystem.

1.4 Scope and Limitations

The scope includes processing the Stack Overflow 2018 Developer Survey, conducting data preprocessing, performing RDD and SQL analyses, and developing supervised classification models. The analysis focuses on career trajectory, education, skills, employment, salary, and developer role segmentation. It does not include real-time deployment, production integration, external datasets, or advanced hyperparameter optimization. The dataset is cross-sectional and self-reported, which limits the ability to infer actual career transitions over time and may introduce reporting bias.

2 Data Selection and Understanding

2.1 Dataset Description

The dataset was obtained from Kaggle and is based on the official Stack Overflow 2018 Developer Survey [1]. It belongs to the software development and workforce analytics domain and includes worldwide responses covering demographics, employment, salaries, skills, and professional experience.

Source Link:

Stack Overflow 2018 Developer Survey

<http://kaggle.com/datasets/stackoverflow/stack-overflow-2018-developer-survey>

- Number of records: 98,855 survey responses.
- Number of columns: 129 attributes.
- Number of files: survey_results_public.csv as the main fact table and survey_results_schema.csv as the reference metadata file.
- Data size: approximately 195.62 MB.
- Time coverage: 2018 cross-sectional survey data.

This dataset is suitable for Spark because its size (~196 MB), complex multi-label categorical columns and the need for combined SQL aggregations, RDD transformations, and machine learning pipelines make Spark's in-memory processing and unified engine efficient even before reaching true big data scale.

2.2 Structure and Schema

The main response file uses Respondent as a unique identifier. There are no foreign keys because the survey is analyzed as a single-table dataset. The schema file was used to understand the meaning of the variables, while the analysis relied on the response file. Table 1 shows the main dataset attributes.

Table 1. Stack Overflow 2018 Developer Survey Dataset Schema.

Attribute	Type	Description	Example Value
Respondent	Integer	Unique identifier for each survey respondent	50823
Country	Categorical	Country where the developer resides	Saudi Arabia
Age	Categorical	Age group of the respondent	25 - 34 years
Gender	Categorical	Self-reported gender	Male
Employment	Categorical	Employment status	Employed full-time
YearsCoding	Categorical	Total years of coding experience	6-8 years
DevType	Multi-label categorical	Developer role(s)	Back-end developer
LanguageWorkedWith	Multi-label categorical	Programming languages used	Python; JavaScript; SQL
ConvertedSalary	Numeric	Annual salary converted to USD	55075
EducationTypes	Multi-label categorical	Education pathways completed	Self-taught; online course

2.3 Initial Data Quality Observations

Initial inspection revealed high missingness in several optional or context-specific variables, including ConvertedSalary, Age, Gender, JobSatisfaction, UndergradMajor, EducationTypes, and LanguageWorkedWith. ConvertedSalary showed strong right skewness and extreme values up to 2,000,000 USD, making outlier detection necessary. No duplicate rows or duplicate Respondent IDs were detected, confirming structural consistency.

2.4 Ethical and Legal Considerations

The dataset is publicly available and anonymized. Each participant is represented by a numeric identifier rather than personally identifiable information. Because the data is publicly shared for analytical and research use, it is acceptable for educational analysis within this course project.

3 Data Preprocessing

All preprocessing steps were implemented in Scala using Apache Spark 4.0.0 through Spark SQL and Spark MLlib APIs [2], consistent with the project's Big Data language convention.

3.1 Cleaning

The preprocessing stage addressed systemic sparsity, occasional missing values, invalid values, salary outliers, and duplicates. A total of 8,073 entirely sparse records were removed because they lacked data across primary features and did not provide useful analytical value. Missing values were handled according to variable type and importance.

Rows with missing DevType were dropped because DevType is a critical target-related variable that cannot be reliably imputed. ConvertedSalary was converted to a valid positive numeric salary and imputed with the mean of valid salaries. Categorical features such as Age, JobSatisfaction, Employment, and Country were filled with their most frequent values. Gender, UndergradMajor, and EducationTypes were filled with "Unknown" to preserve records while indicating missing information. YearsCoding was filled with "0-2 years" as a conservative default. LanguageWorkedWith was inferred using similar respondent characteristics where possible; remaining unknown language records were removed because language usage is central to the project objectives (see Table 2).

Table 2. Data Cleaning Summary.

Issue	Before	After
Missing rows in Country	412	0
Missing rows in DevType	6,757	0
Missing rows in Employment	3,534	0
Missing rows in Salary	51,995	0
Missing rows in Age	34,281	0
Missing rows in Gender	34,386	0 (filled with Unknown)
Missing rows in JobSatisfaction	29,579	0
Missing rows in YearsCoding	5,020	0
Missing rows in LanguageWorkedWith	20,521	0
Missing rows in UndergradMajor	19,819	0 (filled with Unknown)
Missing rows in EducationTypes	30,895	0 (filled with

		Unknown)
Salary outliers	1,418	0
Duplicate respondents	0	0

Salary outlier detection was performed using the Z-score method with a threshold of ± 3 standard deviations. A total of 1,418 extreme salary records were removed to improve the reliability of statistical analysis.

3.2 Integration

The project involved two dataset files, but no technical join was performed in the code. The analysis focused on `survey_results_public.csv`, while `survey_results_schema.csv` was used manually to understand column meanings. Therefore, the final analytical dataset was built from the main responses file only.

3.3 Reduction

The original dataset contained 129 attributes. Feature selection reduced it to 11 problem-relevant core features, including Country, Employment, YearsCoding, Age, Gender, JobSatisfaction, UndergradMajor, EducationTypes, LanguageWorkedWith, ConvertedSalary, and DevType. The Respondent identifier was removed because it has no predictive value. No sampling was applied because the final dataset size was manageable in Spark and using all records preserves representativeness.

3.4 Transformation

Because DevType is a multi-label column with semicolon-separated roles, it was transformed into a single-label target named DevType_segment using rule-based keyword matching. The final role groups were Web, Student/Academic, Other, Mobile, and Data. This step reduced label ambiguity and made supervised classification feasible.

Salary values were converted from ConvertedSalary to numeric Salary_num and log-transformed into Salary_log. Count-based features were engineered from multi-value fields: Languages_count from LanguageWorkedWith and EducationTypes_count from EducationTypes. Categorical variables such as Country, Employment, Gender, UndergradMajor, and DevType_segment were encoded using StringIndexer. Age, YearsCoding, and JobSatisfaction were also ordinal-encoded as Age_encoded, YearsCoding_encoded, and JobSatisfaction_encoded.

After cleaning, reduction, and transformation, the final transformed dataset contained 81,582 rows and 24 columns, including original selected variables and engineered/encoded attributes.

3.5 Summary of Preprocessing Pipeline

The final preprocessing workflow can be summarized as: Raw survey data -> cleaning and imputation -> outlier removal -> feature reduction -> DevType segmentation -> feature engineering and encoding -> final dataset for RDD, SQL, and ML analysis. The same processed dataset was used across the later project phases to ensure consistency.

The complete source code for preprocessing, RDD operations, SQL analysis, and machine learning models is available in the `src/main/scala/` directory of the accompanying GitHub repository (refer to the appendix).

4 RDD Operations

4.1 RDD Design

After converting the preprocessed DataFrame to an RDD, each element represented one survey respondent as a Row object. The main RDD structures included RDD[Row] for all 81,582 respondents, RDD[(String, Double)] for country-salary pairs, RDD[(String, (Double, Int))] for salary aggregation by developer segment, and RDD[String] after flatMap for individual programming language values.

RDDs were useful because several project tasks required custom transformations and fine-grained control. The semicolon-separated LanguageWorkedWith column was naturally decomposed with flatMap. Average salary per developer segment was computed through map -> reduceByKey -> map, making the aggregation logic explicit. Custom high-earner filtering was also easier to express through RDD transformations.

4.2 Key Transformations and Actions

Table 3 shows full summary to RDD Transformations and actions.

Table 3. RDD Analyses Summary.

Analysis	Input	Operations	Output / Insight
Language popularity	LanguageWorkedWith values	flatMap, map, groupByKey, sortBy	JavaScript, HTML, and CSS dominated, while SQL and Python were also widely used.
Salary by role	DevType_segment and Salary_num	map, reduceByKey, map, sortBy	Other and Data roles had the highest average salaries, indicating a premium for specialized skills.
High earners	Salary_num and average salary	filter, count, take	Filtered developers earning above the mean salary and supported further salary analysis.
Data-segment verification	DevType_segment	filter, take	Confirmed that engineered Data labels correctly identify data scientists and analysts.
RDD loading check	Full RDD[Row]	count, first, collect	Verified that 81,582 rows were loaded and that outputs could be collected when small enough.

The following figures (Figure 1-3) provide the required sample outputs for the main RDD analyses. They show the actual printed results produced after applying the corresponding transformations and actions.

```
=====
Analysis 1: Language Popularity
=====
Language           Count
-----
JavaScript          56713
HTML                 55473
CSS                  52816
SQL                  46211
Java                 36916
Bash/Shell           31253
Python               30286
C#                   27473
PHP                  24684
C++                  19706
```

Figure 1. Language Popularity Across the Developer Ecosystem Output.

Interpretation: This analysis answers: "Which programming languages dominate the global developer ecosystem?"

JavaScript ranks first by a substantial margin, reflecting its near-ubiquity across web, mobile, and back-end development. Python and SQL follow closely, driven by the rise of data science, automation, and database-intensive applications. The strong presence of HTML/CSS and Java confirms the continued relevance of traditional web and enterprise development, while emerging languages remain niche

```
=====
Analysis 2: Average Salary by Developer Role
=====
Developer Type      Avg Salary (USD)
-----
Other                $      85621.63
Data                 $      83977.96
Web                  $      80675.20
Mobile               $      77506.71
Student/Academic     $      77282.99
```

Figure 2. Salary Distribution by Developer Role Output.

Interpretation: This analysis answers: "Which developer role is associated with the highest compensation?"

The "Other" category — which includes DevOps, QA, and engineering management — leads in average salary, suggesting that specialized or senior roles command premium compensation. Data scientists and analysts rank second, reflecting the high market demand for data skills. The relatively small salary gap between Web, Mobile, and Student/Academic segments (\$3,000 range) suggests that entry-level and mainstream roles are becoming increasingly comparable in compensation.

```
=====
Analysis 3: High Earners Above Average Salary
=====
Average Salary : $80,315.76
Total Records  : 81,582
High Earners   : 50,358
Percentage     : 61.7%
Sample High Earners:
Country        DevType      Salary (USD)
-----
Kenya          Web          $      97323.10
United States  Web          $      97323.10
United States  Web          $     120000.00
Nigeria        Web          $      97323.10
United States  Web          $     250000.00
```

Figure 3. High Earners and Above-Average Salary Filter Output.

Interpretation: This analysis answers: "What are the most widely used programming languages in the global developer community?"

JavaScript, HTML, and CSS dominate the top three positions, which is consistent with the fact that Web developers form the largest segment in our dataset (55,316 respondents). The strong presence of SQL (4th place) reflects the importance of data querying across all developer roles. Notably, Python ranks 7th overall — but given that Data developers are a small segment (2,371), this indicates Python is also widely adopted beyond data science roles.

These outputs show that the RDD analyses produced concrete results: the language output highlights dominant programming languages, the salary-by-role output confirms the premium for specialized roles, and the high-earner output supports later analysis of salary patterns.

One representative RDD snippet for the role-level salary analysis was:

```
val salaryByDevType = rdd.map(row => (row.getAs[String]("DevType_segment"),
(row.getAs[Double]("Salary_num"), 1)))
.reduceByKey((a, b) => (a._1 + b._1, a._2 + b._2))
.map(x => (x._1, x._2._1 / x._2._2))
.sortBy(_._2, ascending = false)
```

In addition to the three analytical blocks, the RDD phase documented core Spark transformations and actions to demonstrate distributed processing behavior. Map was used to project full records into key-value pairs such as (Country, Salary_num). Filter was applied to isolate high earners and specific role groups. flatMap was important for exploding semicolon-separated language strings into individual language records. groupByKey and reduceByKey were then compared for aggregation, with reduceByKey being more efficient because it reduces values locally before shuffling data across the cluster.

The action operations were used carefully. count() confirmed the dataset size after preprocessing, first() verified the record structure, reduce() computed aggregate salary totals, take() retrieved small samples for validation, and collect() was reserved for compact result sets such as the five developer-segment salary averages. This practice avoids moving large distributed datasets unnecessarily to the driver node.

4.3 Reflection on RDD Usage

RDDs made custom transformations transparent and helped verify the behavior of preprocessing outputs. However, compared with DataFrames and SQL, RDD code was more verbose and required careful handling when collecting results to the driver. For a dataset of this size, Spark managed the computations effectively, but the project still emphasized limiting collect() to small result sets and using aggregations before bringing outputs back to the driver.

5 SQL Operations

5.1 DataFrame and View Creation

The preprocessed dataset was loaded into a Spark DataFrame using spark.read and registered as a temporary SQL view named developer_survey. This enabled declarative analytical queries over the cleaned dataset while preserving compatibility with the previous preprocessing stage.

5.2 Key Analytical Queries

Table 4 shows the SQL query Insights summary.

Table 4. SQL Query Insights Summary.

Query	Natural Language Question	Main SQL Features	Interpretation
Average Salary by Developer Type	What are the average salaries for each developer role?	GROUP BY, AVG, STDDEV, ORDER BY	Other (\$85,622) and Data (\$83,978) roles earned the highest average salaries.
Salary Statistical Summary	What are the key salary distribution measures?	COUNT, DISTINCT, AVG, PERCENTILE,	Mean salary was about \$80,978, median was \$80,000, and salaries varied across 178 countries.

		MIN, MAX	
Educational Pathways to Data Roles	Which undergraduate majors most often lead to Data roles?	CASE WHEN, GROUP BY, HAVING	Mathematics/statistics and social sciences showed higher transitions into Data roles than computer science alone.
Self-Learning vs Formal Education	How does education type relate to salary?	Filtering, aggregation	Developers combining formal education with self-learning earned more than self-learning-only respondents.
Languages by Developer Type	Which languages characterize each role?	String patterns, grouping	Data roles were associated with Python, SQL, and R, while Web roles used JavaScript, HTML, and CSS.
Age and Experience Salary Distribution	How do age and experience relate to salary?	CASE ordering, GROUP BY	Salary increased with experience, peaking at about \$110,684 for 30+ years.
Global Talent Distribution	Which countries have the largest and highest-paid developer populations?	GROUP BY, HAVING, LIMIT	The United States had the largest developer count and high average salaries; India had a large base but lower average salaries.

Representative result (in Figure 4 – 7) excerpts from the original SQL phase are included below to document the outputs used for interpretation.

```
val df = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("stackoverflow_final_transformed.csv")

println("=== Dataset Loaded for SQL Analysis ===")
println(s"Rows: ${df.count()} | Cols: ${df.columns.length}")
println()

// Register as temporary view
df.createOrReplaceTempView("developer_survey")
```

Figure 4. DataFrame and View Creation.

```
=====
QUERY 1: Average Salary by Developer Type
=====
+-----+-----+-----+-----+-----+
|DevType_segment|developer_count|avg_salary_usd|salary_stddev|salary_variance|
+-----+-----+-----+-----+-----+
|Other          |5152           |85621.63      |46058.54     |2.12138929528E9|
|Data           |2371           |83977.96      |50572.15     |2.55754252219E9|
|Web            |55316          |80675.2       |48941.94     |2.39531332356E9|
|Mobile         |4209           |77506.71      |47812.95     |2.28607817333E9|
|Student/Academic|14534          |77282.99      |42113.19     |1.77352108013E9|
+-----+-----+-----+-----+-----+
```

Figure 5. Average Salary by Developer Type Query Result.

QUERY 5: Top Programming Languages per Developer Type

DevType_segment	language	lang_count
Data	Python	1626
Data	SQL	1464
Data	R	1063
Mobile	Java	2761
Mobile	Swift	1878
Mobile	Objective-C	1720
Other	Python	2370
Other	Bash/Shell	2299
Other	HTML	2140
Student/Academic	HTML	10169
Student/Academic	CSS	9537
Student/Academic	JavaScript	9038
Web	JavaScript	43667
Web	HTML	41251
Web	CSS	39767

Figure 6. Top Programming Languages by Developer Type Query Result.

QUERY 7: Global Talent Distribution - Top 15 Countries

Country	developer_count	avg_salary_usd	data_roles_count	pct_data_roles
United States	17994	101358.0	724	4.0
India	11490	72991.0	236	2.1
Germany	5368	77864.0	155	2.9
United Kingdom	5314	88275.0	190	3.6
Canada	2887	78521.0	83	2.9
Russian Federation	2292	60967.0	48	2.1
France	2092	73279.0	86	4.1
Brazil	2027	61160.0	48	2.4
Poland	1766	66366.0	36	2.0
Australia	1701	89998.0	54	3.2
Netherlands	1531	76135.0	43	2.8
Spain	1437	76956.0	42	2.9
Italy	1293	85669.0	34	2.6
Ukraine	1010	67713.0	17	1.7
Sweden	996	74947.0	21	2.1

Figure 7. Global Talent Distribution Query Result.

Together, these excerpts cover SQL view creation, aggregation, ranking, string processing, grouping, and filtering. They also provide visible evidence of the output tables used to support the SQL interpretations summarized above. A representative SQL query used to compute salary by developer type was:

```
SELECT DevType_segment, COUNT(*) AS developer_count,
       ROUND(AVG(Salary_num), 2) AS avg_salary_usd,
       ROUND(STDDEV(Salary_num), 2) AS salary_stddev
FROM developer_survey
GROUP BY DevType_segment
ORDER BY avg_salary_usd DESC;
```

The SQL phase intentionally covered a variety of query types. Aggregation queries measured average salary and salary dispersion. Filtering and HAVING clauses restricted analysis to meaningful groups, such as countries with at least 50 respondents or undergraduate majors with sufficient sample size. CASE expressions were used to construct ordered age and experience groups. These queries provided interpretable summaries that directly informed the machine learning phase by identifying features likely to carry predictive signal.

The strongest SQL-driven modeling signals were DevType_segment, UndergradMajor, Salary_num, YearsCoding, and language-related indicators. The analysis also revealed that class imbalance would need special handling.

5.3 Summary of Insights from SQL

- Specialized roles, especially Other and Data, showed the highest average salaries, validating DevType_segment as a meaningful classification target.
- Educational background influenced career trajectory; quantitative majors such as mathematics and statistics were strong indicators for Data roles.
- Developers with mixed or diverse educational experiences earned 11.4% more than self-learning-only respondents, showing that combining formal education, self-learning, and other learning paths was associated with higher average salaries.
- France showed a relatively high concentration of Data/AI-related roles compared with its developer population size, suggesting a potential regional talent hub for emerging technical roles.
- Salary increased consistently with experience, making Salary_num and YearsCoding_encoded useful predictive signals.

6 Machine Learning Operations

6.1 Problem Formulation

The machine learning task is a multi-class classification problem. The target variable is DevType_segment, a derived categorical label with five classes: Web, Student/Academic, Other, Mobile, and Data. The final dataset contained 81,582 clean records. The target distribution was imbalanced, with Web representing 67.8% of observations and Data only 2.9% (see Table 5).

Table 5. Developer Type Class Distribution.

Label	Class	Count	Share
0	Web	55,316	67.8%
1	Student / Academic	14,534	17.8%
2	Other	5,152	6.3%
3	Mobile	4,209	5.2%
4	Data	2,371	2.9%

Nine features were selected across demographic, professional, educational, and behavioral domains. Categorical columns were indexed and one-hot encoded, while numeric columns were assembled directly. Logistic Regression used StandardScaler because it is gradient-based and sensitive to feature scale, while Random Forest used unscaled feature vectors because tree-based models are scale-invariant. See Table 6 for all ML input Features.

Table 6. ML Input Features.

Feature	Type	Description
Age_encoded	Numeric	Developer age group after ordinal encoding
YearsCoding_encoded	Numeric	Years of coding experience after ordinal encoding
JobSatisfaction_encoded	Numeric	Self-reported job satisfaction score
Employment	Categorical	Employment status
Gender	Categorical	Self-reported gender identity
UndergradMajor	Categorical	Undergraduate field of study
Languages_count	Numeric	Number of programming languages used
EducationTypes_count	Numeric	Number of education types completed

Salary_num	Numeric	Annual salary in USD
------------	---------	----------------------

6.2 Feature Pipeline

The preprocessing pipeline consists of four stages. First, StringIndexer transforms each categorical column (Employment, Gender, UndergradMajor) into numeric indices, mapping each distinct category to an integer. Second, OneHotEncoder converts these indexed values into sparse binary vectors, preventing the model from falsely interpreting categories as ordered numeric values. Third, VectorAssembler combines all numeric features (Age_encoded, YearsCoding_encoded, JobSatisfaction_encoded, Languages_count, EducationTypes_count, Salary_num) with the one-hot encoded vectors into a single dense feature vector required by Spark ML estimators. Fourth, for Logistic Regression only, StandardScaler applies zero-mean and unit-variance scaling to the assembled vector — a necessary step because gradient-based models are sensitive to feature scale, while Random Forest bypasses this stage entirely as tree-based models are scale-invariant.

Excluded features include unique identifiers (Respondent), high-cardinality categoricals (Country), raw multi-label columns (LanguageWorkedWith), and free-text fields, all omitted to reduce noise, prevent overfitting, and maintain pipeline efficiency.

6.3 Model Choice

Two complementary classifiers were used. Logistic Regression provided a linear and interpretable baseline for multi-class prediction through multinomial softmax, with RegParam = 0.01 and ElasticNetParam = 0.5. Random Forest was selected as a non-parametric ensemble able to capture non-linear relationships and feature interactions, using 50 trees, max depth 8, subsamplingRate 0.8, and square-root feature subset strategy.

6.4 Training and Evaluation Setup

The clean dataset was split 70/30 with seed = 42, yielding 57,359 training rows and 24,223 test rows. Class weights were computed on the training set using inverse frequency weighting: $\text{weight} = \frac{\text{total_train_rows}}{(\text{num_classes} \times \text{class_count})}$. This prevented the models from optimizing only for the dominant Web class. Accuracy, weighted F1, and macro F1 were reported, with macro F1 treated as the primary metric because it gives equal weight to minority classes.

6.5 Results

Model evaluation results are summarized in Table 7.

Table 7. Model Evaluation Results.

Model	Accuracy	Weighted F1	Macro F1
Baseline (majority class)	67.76%	-	-
Logistic Regression	41.60%	47.14%	33.73%
Random Forest	52.42%	57.62%	37.53%

Although both trained models had lower accuracy than the majority-class baseline, this result is expected because class weighting forces the model to learn minority classes rather than simply predicting Web for every observation. Random Forest achieved the best macro F1 score (37.53%), outperforming Logistic Regression (33.73%) and providing a more balanced representation across classes.

6.6 Interpretation

Below, Table 8 shows the classes performance.

Table 8. Per-Class Performance.

Class	LR Precision	LR Recall	LR F1	RF Precision	RF Recall	RF F1
Web (0)	0.825	0.343	0.485	0.845	0.509	0.635
Student (1)	0.623	0.680	0.650	0.605	0.707	0.652
Other (2)	0.114	0.289	0.163	0.140	0.219	0.171
Mobile (3)	0.114	0.558	0.189	0.171	0.449	0.248
Data (4)	0.125	0.483	0.199	0.103	0.518	0.172

The following original figures (Figure 8 and 9) add visual evidence for the error analysis and feature interpretation. ROC-AUC was not used because this is a multi-class classification problem; extending AUC to multiple classes requires one-vs-rest approaches.

IT 462 | Big Data Systems

True \ Pred	Web	Student	Other	Mobile	Data
Web	8,349	1,709	1,718	2,172	2,465
Student	683	3,091	102	150	347
Other	402	137	320	320	279
Mobile	327	104	80	563	181
Data	119	70	71	89	375

Figure 8. Random Forest Confusion Matrix.

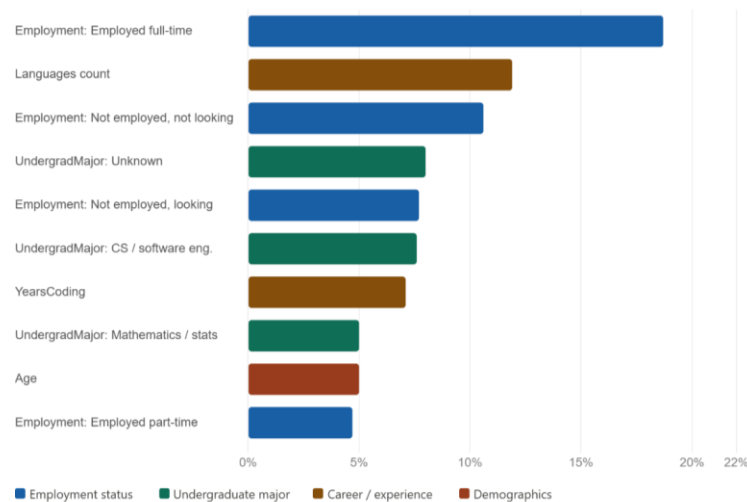


Figure 9. Random Forest Feature Importance.

The confusion matrix shows that the model correctly identifies many Web and Student/Academic records but still confuses minority classes with Web. The feature-importance chart explains why Employment status, Languages_count, and UndergradMajor were central to the Random Forest predictions.

Feature interpretation showed that Employment status was the strongest predictor, followed by Languages_count and UndergradMajor — which aligns with domain expectations, as employment status reflects professional commitment, language count indicates technical breadth, and undergraduate major captures

foundational domain knowledge. The models performed best on the two largest classes (Web, Student/Academic) but struggled with minority classes (Other, Mobile, Data), where many records were misclassified as Web due to class imbalance and genuine feature overlap between developer roles.

The Random Forest model provided a useful trade-off between interpretability and non-linear modeling power. Its improvement in Macro F1 indicates that role prediction benefits from interaction effects that a linear model cannot fully capture. However, the per-class results show that the problem remains difficult: many respondents share similar employment patterns, education histories, and language counts across multiple role categories.

From a domain perspective, the results suggest that broad background features can identify general role families but are not sufficient for fine-grained specialization. For example, a web developer and a mobile developer may both be employed full-time, have similar experience, and use overlapping languages. More detailed technology-specific features would be needed to improve minority-class performance.

6.7 Limitations

- The selected features do not fully separate developer specializations, especially overlapping roles such as Web, Mobile, and Other.
- DevType is self-reported and multi-label in the original survey, so converting it into a single label introduces some noise.
- The survey may over-represent particular regions and developer communities, limiting generalizability.
- Logistic Regression is linear, so it may not capture complex interactions between education, employment, salary, and skills.
- No extensive hyperparameter tuning or external validation dataset was included due to course scope.

7 Conclusion and Future Work

7.1 Summary of Contributions

This project completed an end-to-end big data analysis workflow using the Stack Overflow 2018 Developer Survey. The team selected and understood the dataset, cleaned and transformed raw survey responses, engineered a stable developer-role target, analyzed patterns using RDDs and SQL, and built supervised classification models in Spark ML. The final workflow demonstrates how large-scale survey data can be transformed into useful insights about developer roles, salaries, education, and technical skills.

Key findings include: specialized roles such as Other and Data had the highest average salaries; web developers formed the dominant class in the dataset; quantitative undergraduate backgrounds were strong indicators of Data roles; programming language patterns differed by developer role; and Random Forest produced the strongest macro F1 score among the tested models.

7.2 Reflection

The project reinforced the importance of systematic preprocessing before analysis or modeling. Missing values, multi-label columns, and outliers strongly influenced downstream tasks. Spark RDDs were useful for custom transformations, SQL helped express analytical questions clearly, and MLlib supported a reproducible classification workflow. The main practical challenges involved handling high missingness, converting multi-label roles into a single target, and evaluating imbalanced classification results fairly.

7.3 Future Work

- Add newer Stack Overflow surveys to compare career trends over multiple years.
- Use richer text-based features from DevType and LanguageWorkedWith rather than only count-based indicators.
- Test alternative models such as Gradient Boosted Trees or XGBoost to capture more non-linear interactions.
- Apply advanced imbalance strategies such as SMOTE or class-specific thresholding within the training fold.
- Develop a simple dashboard to present salary, skills, and role trends interactively for recruiters and learners.

8 References

- Kaggle, "Stack Overflow 2018 Developer Survey," Kaggle, [Online]. Available:
1] <http://kaggle.com/datasets/stackoverflow/stack-overflow-2018-developer-survey>.
- Apache Spark, "MLlib: RDD-based API," Apache Spark, [Online]. Available:
2] <https://spark.apache.org/docs/latest/mllib-guide.html>. [Accessed 2026].

9 Appendix

The complete code for all preprocessing, analysis, and model implementation is available and accessible via the following GitHub repository.

Github Link: <https://github.com/iRoseM/stackoverflow-spark-analysis>